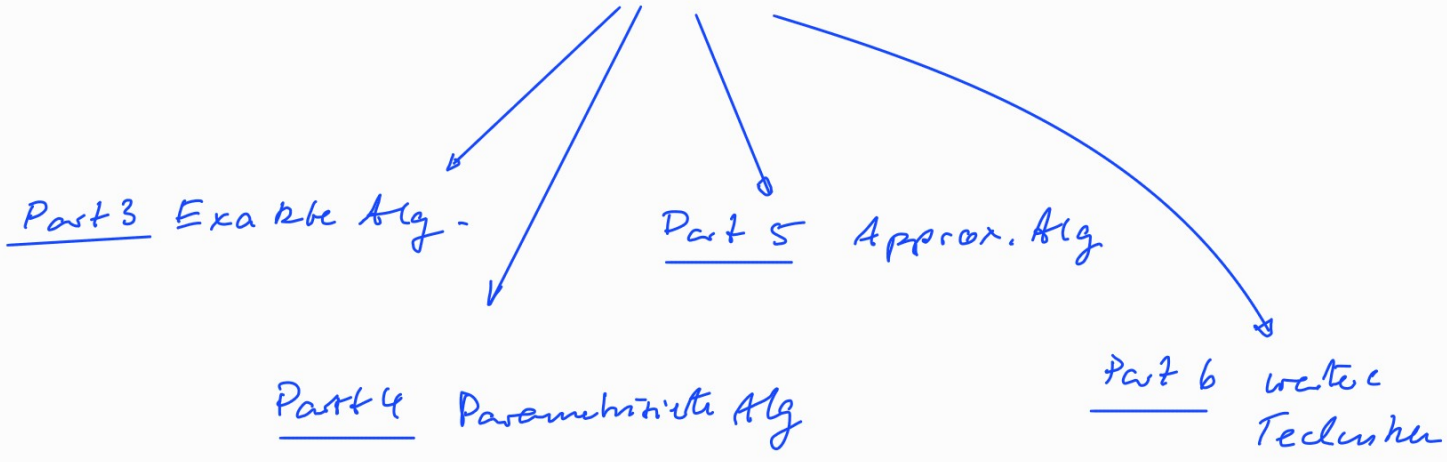
Fahrplan

Part 1 TM, Algorithmen & Entscheidbarkeit

Part 2 Einfach vs schwere Probleme & NP vollständigkeit

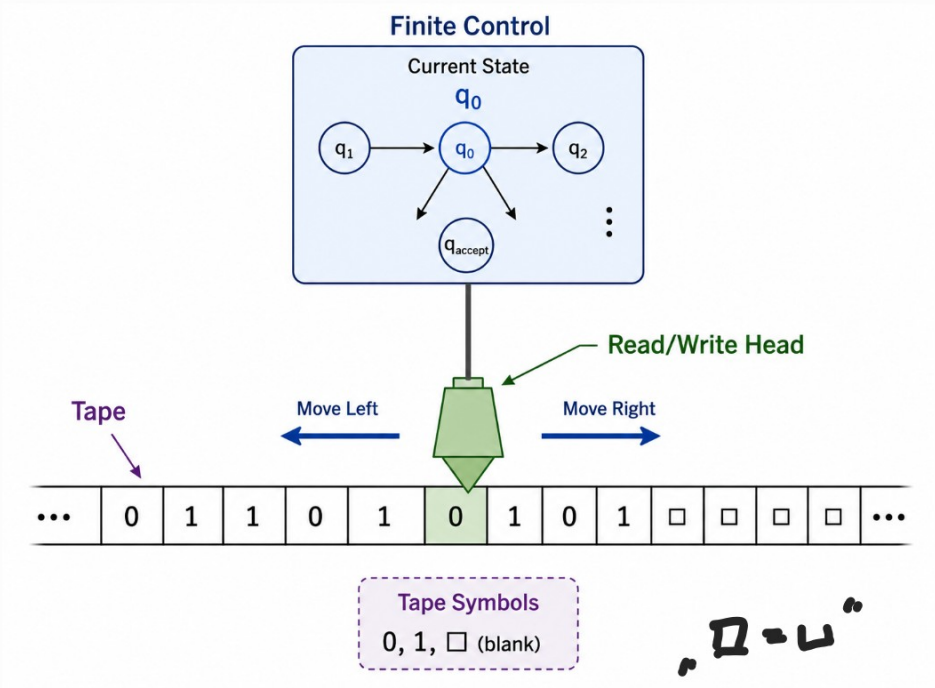


1. Turing Maschinen, Algorithmen & Entscheidbarkeit

In a nutshell: Turing maschinen (TM)

- proposed by Alan Turing 1936
- ähnlich zu endlichen Automaten, aber unendlicher, unrestriktiver Speicher

Schematisch:



- Steuerung (endlich, Start + Zwischen + Reject/accept Zustände)
- unendliches Tape oder Band
- Lese/Schreibkopf → kann 1 Schritt nach rechts oder links gehen

Def:

Σ Alphabet (menge von Buchstaben, Zahlen, Symbolen...)

Σ^* Menge aller endlichen Wörter über Σ einschließlich leeren wort ε

$[\varepsilon w = w \varepsilon = w]$

$L \subseteq \Sigma^*$ Sprache

BSP 1

Wollen TM konstruieren, die feststellt ob gegebenes wort w in $L = \{1b1 \mid b \in \{0\}^*\} \subseteq \{0,1\}^*$ enthalten ist.

($b = \varepsilon$ leeres Wort erlaubt)

Beobachtung TM kann sich nichts "merken", nur Zustände wechseln

i.W. müssten wir folgende Tabelle erstes Symbol "1"
dann nur "0"
dann ein mal "1"
dann nichts = ␣ Blanksymbol.

L-S-Hopf

1 0 0 1 ␣
1 0 0 1 ␣
1 0 0 1 ␣
1 0 0 1 ␣
1 0 0 1 ␣ accept.

1 0 1 0 ␣
1 0 1 0 ␣
1 0 1 0 ␣
1 0 1 0 ␣ reject.

Formal Def TM

Eine TM M ist ein 7-tupel $(Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$
wobei Q, Σ, Γ endliche Menge sind
und:

(1) Q ist die Menge der Zustände

(2) Σ Eingabealphabet welches nicht das Blanksymbol ␣ enthält

(3) Γ Tapealphabet, wobei $␣ \in \Gamma$ & $\Sigma \subseteq \Gamma$

(4) $\delta: Q \times \Gamma \longrightarrow Q \times \Gamma \times \{L, R\}$ Übergangsfunktion (L Links, R Rechts)

(5) $q_0 \in Q$ Start-Zustand

(6) $q_{accept} \in Q$ akzeptierender Zustand (halte Zustand = stop)

(7) $q_{reject} \in Q$ zurückweisender Zustand ($q_{accept} \neq q_{reject}$)
(halte Zustand = stop)

$$\delta: (\underbrace{q_i}_{\in Q}, \underbrace{\text{Symbol1}}_{\in \Sigma}) \mapsto (\underbrace{q_j}_{\in Q}, \underbrace{\text{Symbol2}}_{\in \Gamma}, R/L)$$

Wenn in Zustand q_i das Symbol 1 gelesen wird
dann gehe in Zustand q_j, schreib Symbol 2, & gehe
1 pos. R oder L

"Tabellenform"

δ	Symbol 1
q_i	$(q_j, \text{Symbol2}, R)$

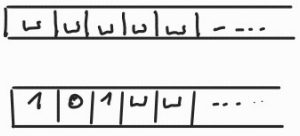
Funktionsweise: $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$

Band/Tape hat unendlich viele Felder $\square$, alle initiiert mit "$\sqcup$".
Können dabei annehmen es gibt "linkestes" Feld

M erhält endliches wort $w = w_1 \dots w_n \in \Sigma^*$ als Input.
auf am weitesten links-liegenden Feld Eintrag w_1

Bsp:

input 101



da $\sqcup \notin \Sigma$ $\Rightarrow$ erstes $\sqcup$-Symbol markiert
Ende von Input.

dann werden die Regeln bzgl der Übergangsfkt abgearbeitet.

KONVENTION: Wenn ↓ L-S-kopf schon am linkesten Rand von Tape & Übergangsfkt sagt "gehe links (L)" dann bleibt ↓ einfach auf dem linken Feld.

Die Berechnung geht solange bis entweder der Zustand q_{accept} oder q_{reject} erreicht ist. (HALTS, Stoppt)
wenn das nicht passiert, läuft TM für immer
(does not HALT, stoppt nicht)

BSP 1 TM für Test $w \in L = \{1b1 \mid b \in \{0\}^*\}$, $\Gamma = \{0, 1, \sqcup\}$

Zustand q_0 = start zustand.

2 Fälle: (1) in Zustand q_0 liest 1 → ok, dann gehe zu Zustand q_1 & verschiebe um 1 position nach rechts

Formal: $(q_0, 1) \longmapsto (q_1, 1, R)$ (das ist was schreiben soll (hier 1 durch 1 "ersetzen"))

(2) In Zustand q_0 liest Symbol $\neq 1$ → will ok, soll rejecten.

diese Symbole liegen alle in Γ

also Symbol = "$\sqcup$" oder "0"

Formal: $(q_0, 0) \longrightarrow (q_{reject}, 0, R)$

$(q_0, \sqcup) \longrightarrow q_{reject}, \sqcup, R)$

[STOP & fertig]

Wenn also $(q_0, 1) \longrightarrow (q_1, 1, R)$ sind wir in Zustand q_1

jetzt müssen wir testen, ob in $w = 1b1$

$b = 0 \ldots 0$ oder $b = \varepsilon$ leeres Wort.

Fälle (1') in Zustand q_1 wird 0 gelesen, dann gehe 1 pos. nach rechts & lies nächstes Symbol

$(q_1, 0) \longrightarrow (q_1, 0, R)$

dürfen in dem Zustand bleiben (kann quasi eine "schleife")

(2') in Zustand q_1 wird 1 gelesen, dann geh 1 pos. Rechts & check ob Symbol dort "$\sqcup$" ist $(q_1, 1) \longrightarrow (q_2, 1, R)$

(3') in Zustand q_1 wird $\sqcup$ gelesen ($\Gamma = \{0, 1, \sqcup\}$) dann reject $(q_1, \sqcup) \longrightarrow (q_{reject}, \sqcup, R)$

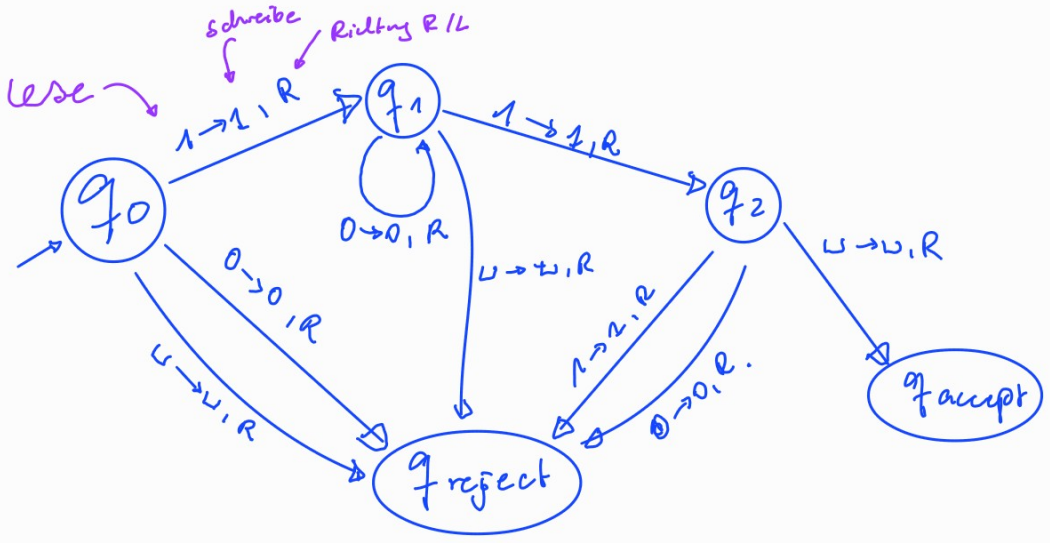
Zustand q_2: muß "$\sqcup$" gelesen werden, sonst reject.

$(q_2, \sqcup) \longrightarrow (q_{accept}, \sqcup, R)$

$(q_2, 1) \longrightarrow (q_{reject}, 1, R)$

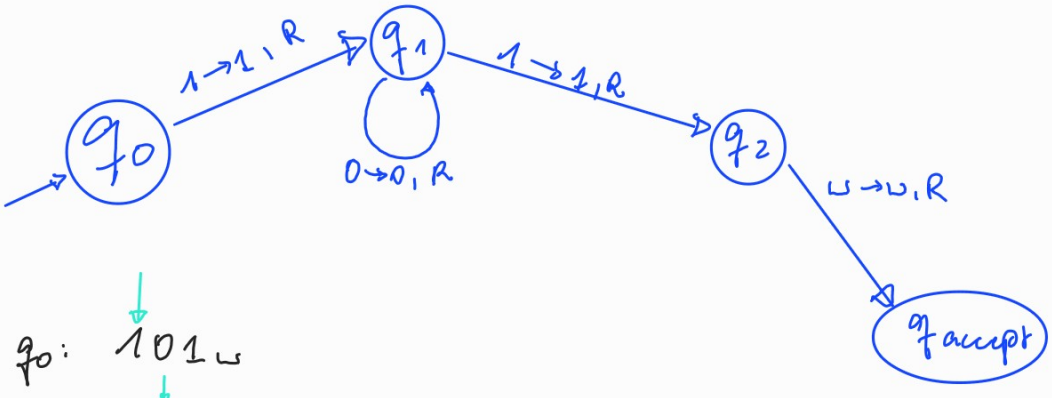
$(q_2, 0) \longrightarrow (q_{reject}, 0, R)$

Zustandsrepräsentation



Vereinfachte Zustandsrepräsentation

nur "accepting" Kanten, alle nicht aufgeführten Übergänge führen direkt zu "reject":



q_0: 1 0 1 ␣
q_1: 1 0 1 ␣
q_1: 1 0 1 ␣
q_2: 1 0 1 ␣
q_{accept}.

DEF: Für TM M, sei $L(M) = \{w \mid M \text{ accepts } w\}$ die Sprache von M, auch die Sprache die M erkennt

DEF:

$L \subseteq \Sigma^*$ ist TM-akzeptierbar $\Leftrightarrow$ $\exists$ TM M mit $L = L(M)$

(oder auch TM-erkennbar)

TM akzeptiert, rejects oder läuft für immer.

Wir wollen i.d.R. TMs die immer stoppen, d.h. entweder den Zustand q_{accept} oder q_{reject} erreichen.

DEF: TMs die auf jeder Eingabe halten (also q_{accept} oder q_{reject}) heißen Entscheider

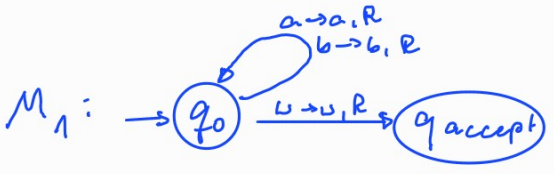
Eine Sprache L heißt entscheidbar (auch rekursive (aufzählbar))

$\Leftrightarrow$ Entscheider M gibt sd $L(M) = L$

[in anderen Worten
wenn es eine TM M gibt, die für alle $w \in L$ im Zustand q_{accept} hält & f.a. $w \notin L$ im Zustand q_{reject} hält.]

Bsp 1: $L = \{1b1 \mid b \in \{0\}^*\}$ ist entscheidbar (siehe oben)

Bsp 2: $L = \{w \mid w \in \{a,b\}^*\}$ ist entscheidbar.



$L(M_1) = L$

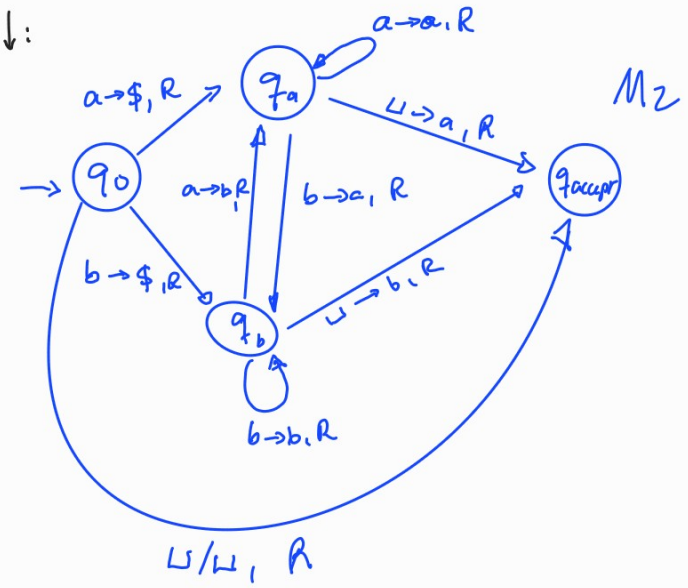
man kann TMs aber auch anders um Wörter zu modifizieren.

gibt es TM die L entscheiden & wo finales Wort in q_{accept} Zustand der Form $\$w$ $\forall w \in L$?

C.i.w. Algo-design mit super-beschränkten Regeln

Antwort JA: f (nächste Seite)

↓:



q_0	a	b	a	a	⊔	..
q_a	$	b	a	a	⊔	..
q_b	$	a	a	a	⊔	..
q_a	$	a	b	a	⊔	..
q_a	$	a	b	a	⊔	..
q_{accept}	$	a	b	a	a	_

$L(M_2) = L(M_1)$
aber unterschiedliches output.

- Muß man zum erkennen von Sprachen, schreiben können?

JA, nicht immer, aber oft:

$L = \{ 0^n 1^n, n \in \mathbb{N} \}$

↓

hier muß man sich "merken" wie "oft" 0 & 1 vorkamen
d.h. man braucht (unbegrenzten) Speicher

[nicht regulär (wegen pumping-lemma, d.h. kann nicht mit diskret. endlichen Automat (DEA) erkannt werden

DEA = TM mit nur Rechts & nicht schreiben
$\delta: Q \times \Gamma \to Q$

IDEE: TM:

- suche erste nicht-mark. 0 & markiere mit X
- suche erste nicht-mark 1 & markiere mit Y
- wdh.
- akzept gdw. alle 0 & 1 (paarw.) markiert wurden

(überschreiben mit X, Y = i.W. unbeschränkt viele Zustände zu speichern.

Variationen von TM's

- single tape vs multi-tape TM (mehr Tapes)
- unfinite an beiden "Enden"
- non-deterministisch

multitape TM hat $k \geq 1$ tapes & entsprechende L-S-köpfe für jedes Tape.
die Übergangsfkt ist dann der Form

$$\delta : Q \times \Gamma^k \longrightarrow Q \times \Gamma^k \times \{L, R, S\}^k$$

(stay)

Q: More powerful? → NO:

Thm: Jede multitape TM kann mittels einer single-tape TM ausgedrückt werden.

Beweis: (Idee) [Anhang - für Interessierte]

Cor: Eine Sprache L ist TM-erkennbar (bzw. entscheidbar)
$\Leftrightarrow$ es gibt eine multitape TM mit unendlichen Enden an beiden Seiten die L erkennt (bzw. entscheidet).

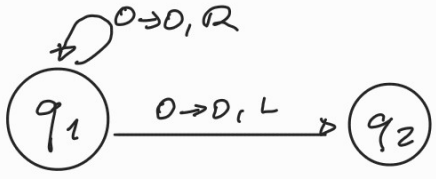
deterministische TM's (wie bisher behandelt)

deterministisch = wenn TM in bestimmten Zustand ist & bestimmtes Symbol liest, ist nächste Zustand eindeutig definiert.

Nicht-deterministische TM

nicht-determinismus ist eine Verallgemeinerung von determinismus. (determ ⇒ nicht-determ)

M:



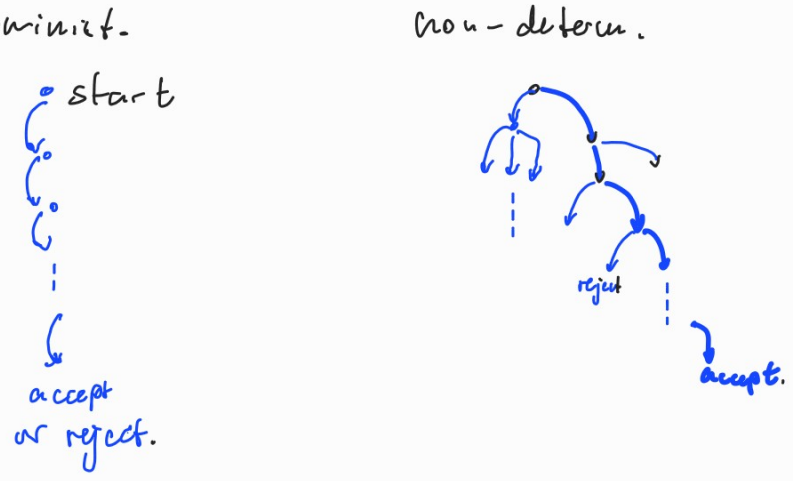
hier kann Symbol 0 zu schleife $q_1 \to q_1$ führen oder zu Zustand q_2.
was sollen wir in diesem Zustand q_1 jetzt tatsächlich tun?

↓

nach dem im Zustand q_1 die '0' gelesen in TM M → TM M spaltet sich in Kopien von sich selbst auf & verfolgt alle Möglichkeiten parallel (jede Kopie verfolgt dann einen der Wege & wiederholt entsprechend letzten schritt).

wenn es dann in einem der Kopien zu einem akzept. Zustand kommt, dann akzeptiert TM den Input.

IDEE: Entscheidungsbaum



Formal: wie TM, aber mit Übergangsfkt

$$\delta : Q \times \Gamma \longrightarrow \mathcal{P}(Q \times \Gamma \times \{L, R\})$$

[$\mathcal{P}$ = potenzmenge]

Thm. Jede nicht-determ. TM kann mittels einer deterministischen TM ausgedrückt werden.

Beweis: (idee) [Anhang - für Interessierte]

Cor: Eine Sprache L ist TM-erkennbar (bzw. entscheidbar) $\Leftrightarrow$ es gibt eine nicht-det. TM die L erkennt (bzw. entscheidet.)

Q: warum das alles?

A: um den Begriff "Algorithmus" zu definieren.

"intuitiv", ist Alg eine endliche, eindeutig beschriebene Folge von (mechanischen) Schritten, die man nach festen Regeln ausführen kann, um ein Problem zu lösen.

Bspw:

```
sum (1...n)
  S=0
  FOR (i=1..n)
    S ← S+i
  return S
```

oder

```
sum (1..n)
  return n(n+1)/2
```

problem: der "intuitive" Begriff erlaubt es nicht zu zeigen, daß es keinen Alg. für ein Problem gibt, dafür braucht man klare formale definition.

um zu zeigen es gibt keinen Alg für Problem P muß man alle denkbaren Verfahren ausschließen. ⇒ wenn man ein "universelles" Verfahren hat, welches alle denkbaren Verfahren umfasst, muß man nur für P zeigt daß es kein "universelles" Verfahren gibt.

universelles Verfahren : TM.

Church-Turing These [Alonzo Church, Alan Turing, 1936]

intuitive Notation von Algorithmus (Berechenbarkeit)	=	TM algorithmus (Berechenbarkeit)

⇒ TMs geben uns präzise Def von Algorithmus

⇒ Q: Was ist das "richtige" Level der Beschreibung eines Algorithmus (TMs mühsam)

↓

Hier: "high-level" Beschreibung

Für TM's: input = string

"high-level" Alg: input = object O (bsw. graph, ein Tableau)

im Computer wird Object O (oder versch. Objekte $O_1 \dots O_n$)

bzw. in String $\langle O \rangle$ (oder $\langle O_1, \dots, O_n \rangle$) übersetzt

↓

dann endliche Schritte von eindtg. Anweisungen

↓

ggf: output Object O' oder $O'_1, \dots, O'_\ell$

Bsp 1: $L = \{1b1 \mid b \in \{0,1\}^*\}$ ist entscheidbar mittels TM (siehe oben)

"high-level" Alg:

Input: $s = s_1 \dots s_k$ (string), $k \geq 2$

IF $s_1 = 1$ & $s_k = 1$ & $s_i = 0 \; \forall i \in \{2, \dots, k-1\}$

RETURN TRUE

ELSE RETURN FALSE

Können von hier an Begriff TM & Algorithmus synonym verwenden

Entscheidbarkeit

Gibt es Probleme, die man nicht mittels Algorithmen lösen kann?

(Wenn man weiß das Problem algorithmisch nicht lösbar ist, weiß man daß man Problem restriktieren muß bevor man irgendeine algor. Lsg finden kann.)

wdh: Algor. (= TM) Fälle:
→ stop in akzept. Zust.
→ stop in reject Zust
→ läuft für immer.

Entscheider sind TM's die auf jeder Eingabe halten (entweder q_accept, q_reject)

Entscheider M mit $L = L(M) = \{w \mid M \text{ akzept } w\}$ entscheiden L

DEF Ein **Entscheidungsproblem** ist ein Problem, für das jede Instanz (= konkrete Eingabe) entweder JA (true) oder NEIN (false) als Antwort hat.

Entscheidungsprobleme können auf natürliche Art & Weise als Sprache L_D aufgefasst werden: $L_D = \{w \mid w \text{ ist JA-Instanz von } D\}$

Bsp1: als Entscheidungsproblem: Ist w der Form $1b1$, $b \in \{0\}^*$? [hat klar JA oder NEIN Antwort]

Instanz: alle Wörter $w \in \{0,1\}^*$

Ja-Instanz: $L_{Bsp1} = \{1b1 \mid b \in \{0\}^*\}$

DEF Ein Entscheidungsproblem D heißt algorithmisch lösbar / entscheidbar
$\Leftrightarrow$ es gibt Entscheider M m.T. $L(M) = L_D$

Gibt es Entscheidungsprobleme die nicht algor. lösbar sind?

Ja! Insbesondere gibt es:
deutlich mehr Entsch.probl. als Algorithmen zum Lösen von diesen Entsch.probleme!

WARUM? : Algorithmus = endliche Sequenz von Buchstaben
= endliches Wort.

(Bsp Pythonprog incl. TABS, Leerzeilen, Zeilenumbrüche).

Jedes endl. Wort wird in Computern intern in endlichen Binärstring umgewandelt

$\Rightarrow$ Alg $\hat{=}$ endlicher Binärstring

$\Rightarrow$ Menge aller Alg $\subseteq \{0,1\}^* \hat{=}$ Menge aller endlichen Wörter über $\{0,1\}$

$\{0,1\}^*$ ist abzählbar unendlich : $\varepsilon, 0, 1, 00, 01, 10, 11, 000 \ldots$
$w^1\ w^2\ w^3\ w^4\ w^5\ w^6\ w^7 \ldots$
nach Länge sortiert & innerhalb gleicher Länge lexicogr. ordnen $\Rightarrow$ abzählbar.

$\Rightarrow$ können $\{0,1\}^*$ schreiben als $\{w^1, w^2, w^3, \ldots\}$ mit $w^1 = \varepsilon$, $w^2 = 0$, $\vdots$

(alternativ, abzählbar: für jede Länge n gibt es 2^n also endl. viele Wörter
$\Rightarrow B = \bigcup_{n \geq 0} \{0,1\}^n$ abzählbare Vereinigung endlicher Mengen
$\Rightarrow$ B abzählbar)

$\Rightarrow$ $\#\text{Algor.} \leq |\{0,1\}^*| = |\mathbb{N}|$

genauso wie Algo. kann man Entscheidungsprobleme binär codieren.

Bsp: Entscheidungsprobleme

⇝ D: Ist gegebene Zahl gerade?

$n \longrightarrow$ binärcodiert $\longrightarrow$ checke letztes Bit,

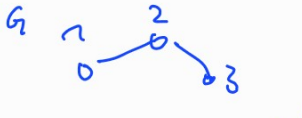
d.h.: 00, 100, 1010 Ja Inst.
01, 101, 1111 NEin Inst.

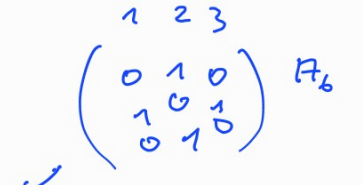
$L_D = \{ w \in \{0,1\}^* \mid w \text{ endet auf } 0 \}$.

(Klar, D ist entscheidbar)

⇝ D: Ist gerg. gerepl zushgd?

Adj-matrix





& bring: $(010\ 101\ 010) = w_G$

$L_D = \{ w_G \mid G \text{ zushgd} \} \subseteq \{0,1\}^*$.

Für test ob G zushgd gibt es Algorithmen (bspw BFS) die mit Eingabe G feststellen ob G zushg oder nicht und die in polynom. terminieren ($\hat{=}$ „intuitiver" Alg, den man auch als TM formulieren kann.

Insbesondere gibt es Entscheider für L_D. $\Rightarrow$ D ist entscheidbar.

Also: • Entscheidungsprobleme verlangen JA/NEIN Antwort

• OB diese Antwort algorithmisch berechnet werden kann ist eine völlig andere Frage!

Entscheidungsproblem $D \hat{=}$ Sprache $L_D \subseteq \{0,1\}^*$
die JA instanzen

$\Rightarrow \forall w \in \{0,1\}^*$: $w \in L_D$ or $w \notin L_D$
(JA) (NEIN)

Haben: $\{0,1\}^* = \{w^1, w^2, w^3, w^4, w^5, \dots\}$
(geordnet)

In dieser Reihenfolge können wir für jedes Entscheidungsproblem D einen Vektor s_D bauen:

Eingabe:	w^1	w^2	w^3	w^4	w^5	...
Antwort	J	N	J	J	N	...
$s_D =$	1	0	1	1	0	...

Bsp: D: Ist Zahl k gerade?

$L_D = \{w \mid w \in \{0,1\}^*, \text{letztes Bit } 0\}$

w^i:	ε	0	1	00	01	10	11	...
s_D	1	1	0	1	0	1	0	...

• $s_D = s_{D'} \Leftrightarrow L_D = L_{D'}$

$L_D = L_{D'} \Leftrightarrow D \sim D'$ bei gleicher Modierung

Bsp: D: ist 6 zusammenhängend?
D': gibt es für alle Knoten u, v in b einen uv-Pfad?

aber $L_D = L_{D'}$ hängt von gewählter Modierung ab.

$L_{adjmatrix} \neq L_{adj-Liste}$

$\Rightarrow$ D (genauer L_D) ist durch unendliche Binärstring $s_D = b_1 b_2 b_3 \ldots$ codiert, wobei $b_i = \begin{cases} 0, & w_i \notin L_D \\ 1, & w_i \in L_D \end{cases}$

$\Rightarrow$ Menge aller Entsch.problem = Menge aller unendl. Bs: nützl. strings. deren Kardinalität ist $|\mathbb{R}|$. (↗ Analysis) Diagonal Arg. Cantor

Algorithmus (end. Binärstring) vs. Entsch.problem (unendl. JA/NEIN Folge (= Binärstring))

$\#\text{Alg} \leq |\mathbb{N}|$

$\#\text{Entsch.prob} = |\mathbb{R}|$

Da $|\mathbb{R}| >> |\mathbb{N}|$ gibt es viel mehr Entsch.probleme als Algorithmen für diese Lsg.

$\Rightarrow$ sehr viele Entscheid.probl. sind nicht alg. lösbar!

Konkretes Bsp HALTE Problem:

Finde Algorithmus der entscheidet ob anderer Algor. mit gegeben Input terminiert.

$L_{HALT} = \{ \langle M, w \rangle \mid M \text{ hält mit input } w \}$

Alg mit input w.

HALT: input: $\langle M, w \rangle$

Wenn M mit input w hält dann RETURN TRUE (accept)

Sonst RETURN FALSE (reject)

L_{HALT} is TM-erkennbar, aber nicht entscheidbar, denn wenn M für immer läuft, kann das nicht mit HALT entschieden werden

Bspw: M: input: ε = leeres Wort/"mix"

WHILE(HALT(A))

Fälle: M terminiert

⇒ HALT($\langle M, \varepsilon \rangle$) returned TRUE

⇒ WHILE in M läuft für immer

⇒ M terminiert nicht ↯

M terminiert nicht

⇒ HALT($\langle M, \varepsilon \rangle$) returned FALSE

⇒ WHILE in M stops

⇒ M terminiert ↯

⇒ HALT kann nicht existieren!

Zusammenfassung

- TM's liefern die Grundlage für den Begriff "Algorithmus"
- Für TM M ist $L(M) = \{w \mid M \text{ akzeptiert } w\}$
- Sprache L ist TM-erkennbar (oder TM-akzeptierbar) wenn es TM M gibt mit $L = L(M)$
- Sprache L ist entscheidbar, wenn es TM M gibt die auf jeder Eingabe hält & $L = L(M)$.
- Entscheidungsproblem D = Problem wo alle Instanzen JA oder NEIN beantwortet werden können.

  L_D = Menge der JA-Instanzen von D

  D heißt algor. lösbar, wenn L_D entscheidbar ist.

es gibt deutlich mehr Entscheidungsprobleme D die nicht algor. lösbar sind.

Jetzt Fokus auf algor. lösbare Probleme.

— ANHANG 6 —

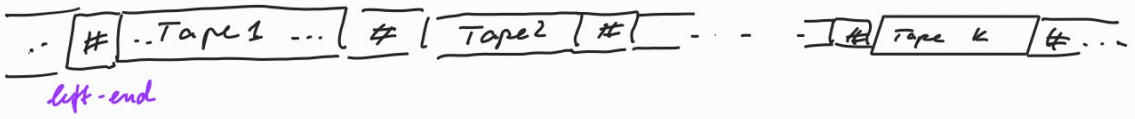
Thm: Jede multitape TM kann mittels einer single-tape TM ausgedrückt werden.

Beweis: (Idee) Sei M multitape TM. mit k tapes

Ziel: simuliere M durch single-tape TM S.

(↗ Beschreibung)

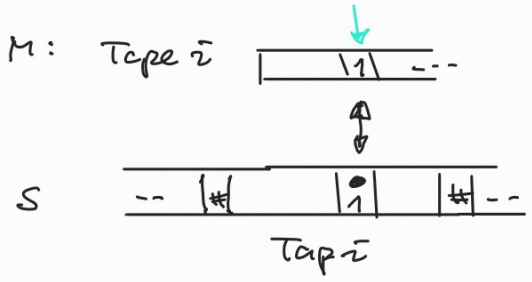
S muss "infos" der k tapes von M auf 1 tape speichern.



Separiere "Inhalt" der Tapes mit '#'

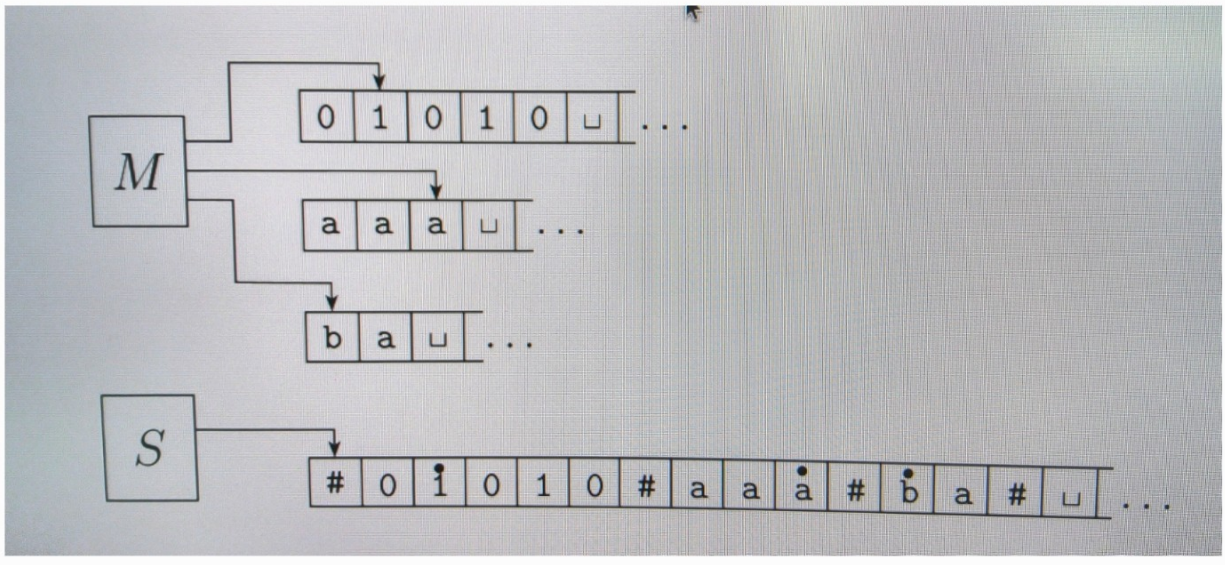
Zudem muß S auch Kontrolle über die k R-S-Köpfe behalten.

→ kann man machen indem



add dot • "(•, 1)" um Stelle zu markieren

(•, 1) just new "symbol" added to Γ



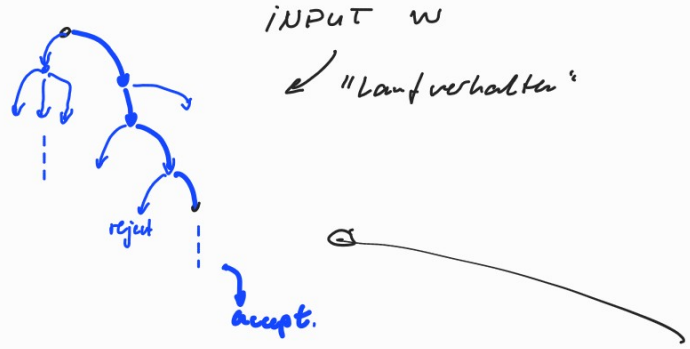
- Start scan from 1st # to last # um Symbole unter "dotted" Köpfe zu bestimmen.
- dann 2. pass um alle "dotted" Zustände entsprechend anzupassen.
  wenn z.B. Tape: aaa# ⇝ aaab#
  ↗ extra länge
  dann shifte alle position entsprechend wie in Bsp 2.

/□

A2

Thm. Jede nicht-determ. TM kann mittels einer deterministischen TM ausgedrückt werden.

Beweis: (Idee) (Berechenbarkeit) non-determ. TM mit INPUT w



"Baue" determ TM D so D diesen Suchbaum durchgeht & nach akzept. Zustand überprüft.

[Depth First Search (DFS, Tiefensuche) nicht gut da es "unendliche" Pfade geben kann]

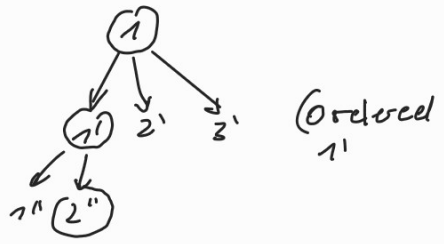
also BFS (Breadth-first search, Breitensuche) garantiert, daß D auch akzept. Zustand nach endlichen Schritten findet, wenn es einen akzept. Zust. gibt.

D hat dann 3 tapes (kann man mittels single tape TM simulieren)

| | | |

initial :
- input tape = input tape von N
- T_2 & T_3 leer

dann start at root (Zustand q_0)
entscheide entsprechend BFS
für einen nächsten Knoten (Zustand)
im Baum.



(ordered
1'

kann man so bauen,
β D anhält, wenn

Algorithmen und Komplexität

1. Turingmaschinen, Algorithmen und Entscheidbarkeit
(Sammlung zentraler Definitionen)

1 Alphabete, Wörter und Sprachen

Definition 1 (Alphabet und Sprache)**.** Ein *Alphabet* Σ ist eine endliche Menge von Symbolen. Ein *Wort* über Σ ist eine endliche Folge von Symbolen aus Σ. Die Menge aller endlichen Wörter über Σ, einschließlich des *leeren Wortes* ε, wird mit Σ^* bezeichnet. Eine *Sprache* über Σ ist eine Teilmenge

$$L \subseteq \Sigma^*.$$

2 Turingmaschinen

Definition 2 (Deterministische Turingmaschine)**.** Eine (deterministische Einband-) *Turingmaschine* ist ein 7-Tupel

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}}),$$

wobei

(1) Q eine endliche Menge von Zuständen ist,
(2) Σ das endliche Eingabealphabet mit $\sqcup \notin \Sigma$ ist,
(3) Γ das endliche Bandalphabet mit $\Sigma \subseteq \Gamma$ und $\sqcup \in \Gamma$ ist,
(4) $\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R\}$ die Übergangsfunktion ist,
(5) $q_0 \in Q$ der Startzustand ist,
(6) $q_{\text{accept}} \in Q$ der akzeptierende Haltezustand ist und
(7) $q_{\text{reject}} \in Q$ der zurückweisende Haltezustand ist, mit $q_{\text{accept}} \neq q_{\text{reject}}$.

Die Übergangsregel $\delta(q, a) = (p, b, R)$ bedeutet, dass die Maschine im Zustand q das Zeichen a durch b ersetzt, nach rechts geht und in den Zustand p wechselt. Analog bezeichnet L eine Bewegung nach links.

Konvention zur Berechnung. Zu Beginn steht die Eingabe am linken Bandrand, der Kopf liest das erste Eingabesymbol (bei ε ein Blank), und alle übrigen Felder sind anfangs leer. Eine Linksbewegung am linken Bandrand lässt den Kopf auf demselben Feld stehen. Die Berechnung endet beim Erreichen von q_{accept} oder q_{reject}; andernfalls kann sie unendlich weiterlaufen. Übergänge aus den beiden Haltezuständen werden nicht ausgeführt.

Definition 3 (Von einer Turingmaschine erkannte Sprache)**.** Für eine Turingmaschine M mit Eingabealphabet Σ heißt

$$L(M) = \{w \in \Sigma^* \mid M \text{ akzeptiert die Eingabe } w\}$$

die *von M erkannte Sprache* (auch: *Sprache von M*).

3 Erkennbarkeit und Entscheidbarkeit

Definition 4 (TM-Erkennbarkeit)**.** Eine Sprache $L \subseteq \Sigma^*$ heißt *TM-erkennbar* (auch: *TM-akzeptierbar*), wenn es eine Turingmaschine M mit $L(M) = L$ gibt.

Definition 5 (Entscheider)**.** Eine Turingmaschine, die auf *jeder* Eingabe hält, heißt *Entscheider*.

Definition 6 (Entscheidbare Sprache)**.** Eine Sprache $L \subseteq \Sigma^*$ heißt *entscheidbar*, wenn es einen Entscheider M mit $L(M) = L$ gibt. Das heißt: M akzeptiert jedes $w \in L$ und verwirft jedes $w \notin L$, jeweils nach endlich vielen Schritten.

4 Varianten von Turingmaschinen

Definition 7 (Mehrband-Turingmaschine)**.** Eine *k-Band-Turingmaschine* besitzt $k \geq 1$ Bänder mit jeweils einem eigenen Lese-/Schreibkopf. Ihre Übergangsfunktion hat die Form

$$\delta : Q \times \Gamma^k \longrightarrow Q \times \Gamma^k \times \{L, R, S\}^k,$$

wobei S bedeutet, dass der betreffende Kopf auf seinem Feld stehen bleibt.

Definition 8 (Determinismus und Nichtdeterminismus)**.** Eine Turingmaschine heißt *deterministisch*, wenn in jeder Konfiguration höchstens ein nächster Berechnungsschritt möglich ist. Bei einer *nichtdeterministischen Turingmaschine* können mehrere Übergänge zur Auswahl stehen. Die Übergangsfunktion kann formal als

$$\delta : Q \times \Gamma \longrightarrow \mathcal{P}(Q \times \Gamma \times \{L, R\})$$

beschrieben werden. Eine nichtdeterministische Turingmaschine *akzeptiert* eine Eingabe genau dann, wenn *mindestens ein* Berechnungspfad akzeptiert.

5 Algorithmen und Entscheidungsprobleme

Begriffserklärung (Algorithmus). Ein Algorithmus ist eine endliche, präzise Beschreibung eines Verfahrens, dessen elementare Schritte mechanisch ausführbar sind. Die Vorlesung verwendet Turingmaschinen als formales Berechnungsmodell für diesen intuitiven Begriff.

Definition 9 (Entscheidungsproblem)**.** Ein *Entscheidungsproblem* ist ein Problem, für das jede Instanz genau eine Antwort aus $\{\text{JA}, \text{NEIN}\}$ besitzt.

Definition 10 (Sprache eines Entscheidungsproblems)**.** Werden die Instanzen eines Entscheidungsproblems D durch Binärwörter kodiert, so ist die zugehörige Sprache

$$L_D = \{w \in \{0,1\}^* \mid w \text{ kodiert eine JA-Instanz von } D\}.$$

Somit entspricht die Entscheidung über eine kodierte Instanz der Frage, ob das zugehörige Wort in L_D liegt.

Definition 11 (Algorithmisch lösbares Entscheidungsproblem)**.** Ein Entscheidungsproblem D heißt *algorithmisch lösbar* (*entscheidbar*), wenn es einen Entscheider gibt, der seine Sprache L_D entscheidet.

Hinweis: es gibt **deutlich** mehr Entscheidungsprobleme als Algorithmen um sie zu lösen. Somit sind sehr viele Entscheidungsprobleme nicht algorithmisch lösbar.